

```
    ❸ newBalance = deposit(userDepositAmount, userPassword)
    if newBalance is not None:
        print('Your new balance is:', newBalance)

--- snip calls to appropriate functions ---

print('Done')
```

Listing 1-3: Bank simulation for one account with functions

In this version, I’ve built a function for each of the operations that we identified for a bank account (create ❶, check balance ❷, deposit ❸, and withdraw ❹) and rearranged the code so that the main code contains calls to the different functions.

As a result, the main program is much more readable. For example, if the user types `d` to indicate that they want to make a deposit ❺, the code now calls a function named `deposit()` ❸, passing in the amount to be deposited and the account password the user entered.

However, if you look at the definition of any of these functions—for example, the `withdraw()` function—you’ll see that the code uses `global` statements ❹ to access (get or set) the variables that represent the account. In Python, a `global` statement is only required if you want to change the value of a global variable in a function. However, I am using them here to make it clear that these functions are referring to global variables, even if they are just getting a value.

As a general programming tenet, functions should *never* modify global variables. A function should only use data that is passed into it, make calculations based on that data, and potentially return a result or results. The `withdraw()` function in this program does work, but it violates this rule by modifying the value of the global variable `accountBalance` ❻ (in addition to accessing the value of the global variable `accountPassword`).

Implementation 3—Two Accounts

The version of the bank simulation program in [Listing 1-4](#) uses the same approach as [Listing 1-3](#) but adds the ability to have two accounts.